

# 轮趣科技

## MR20 毫米波雷达上位机软件及 ROS 使用教程

推荐关注我们的公众号获取更新资料



版本说明:

| 版本   | 日期         | 内容说明  |
|------|------------|-------|
| V1.0 | 2025/09/20 | 第一次发布 |
|      |            |       |

网址: [www.wheeltec.net](http://www.wheeltec.net)

# 目录

|                              |    |
|------------------------------|----|
| 1. 上位机软件 .....               | 3  |
| 1.1 安装 CH9102 芯片驱动 .....     | 3  |
| 1.2 连接使用上位机软件 .....          | 4  |
| 1.3 CH9102 信息修改 .....        | 9  |
| 2. Ubuntu .....              | 10 |
| 2.1 Ubuntu CH9102 驱动安装 ..... | 10 |
| 2.2 ROS1_SDK 包 .....         | 12 |
| 2.3 ROS2_SDK 包 .....         | 16 |

# 1. 上位机软件

## 1.1 安装 CH9102 芯片驱动

### ① 解压资料中的“CH343SER（CH9102 的 windows 驱动）.zip”文件

选中资料包中驱动软件及工具目录下对应的驱动压缩包，进行解压使用。



图 1-1-1 解压 CH9102 驱动压缩包

### ② 安装 CH9102 驱动

双击驱动包中的 SETUP.EXE 文件打开，然后点击安装按钮，等待安装完成后会提示驱动安装成功。

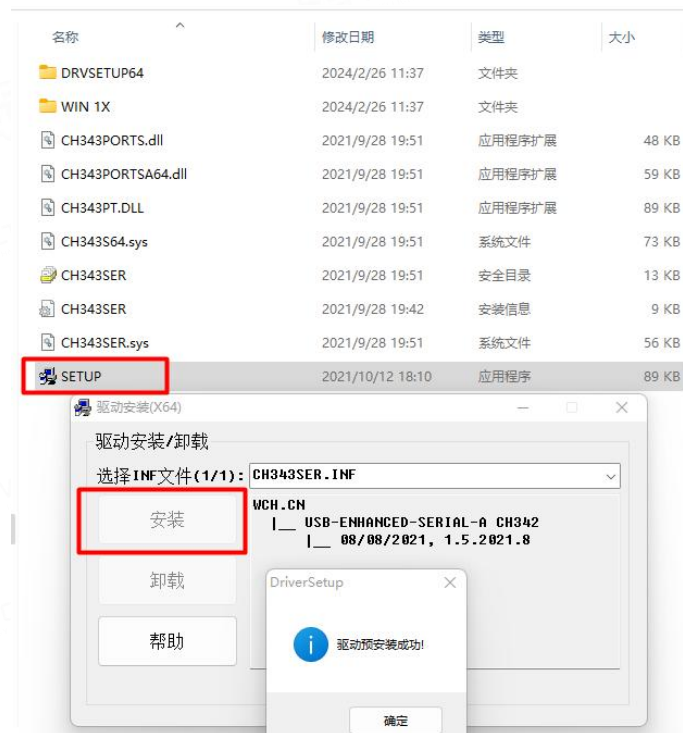


图 1-1-2 驱动安装

## 1.2 连接使用上位机软件

### ① 雷达接线

一开始到手，我们会有四个部分：毫米波雷达本体，毫米波雷达串口线，USB 串口模块，Type-C 线。按顺序连接起来。

### ② 连接电脑运行上位机软件

将 Type-C 线接到电脑的 USB 口，在 windows 中右键此电脑→属性→设备管理器→端口，找到 CH9102 的端口名，其后面对应的 COM 即为毫米波雷达在 windows 中的串口号。



图 1-2-2 查看串口号

解压 windows 上位机软件中的压缩包，得到以下文件夹。



图 1-2-3 解压上位机软件压缩包

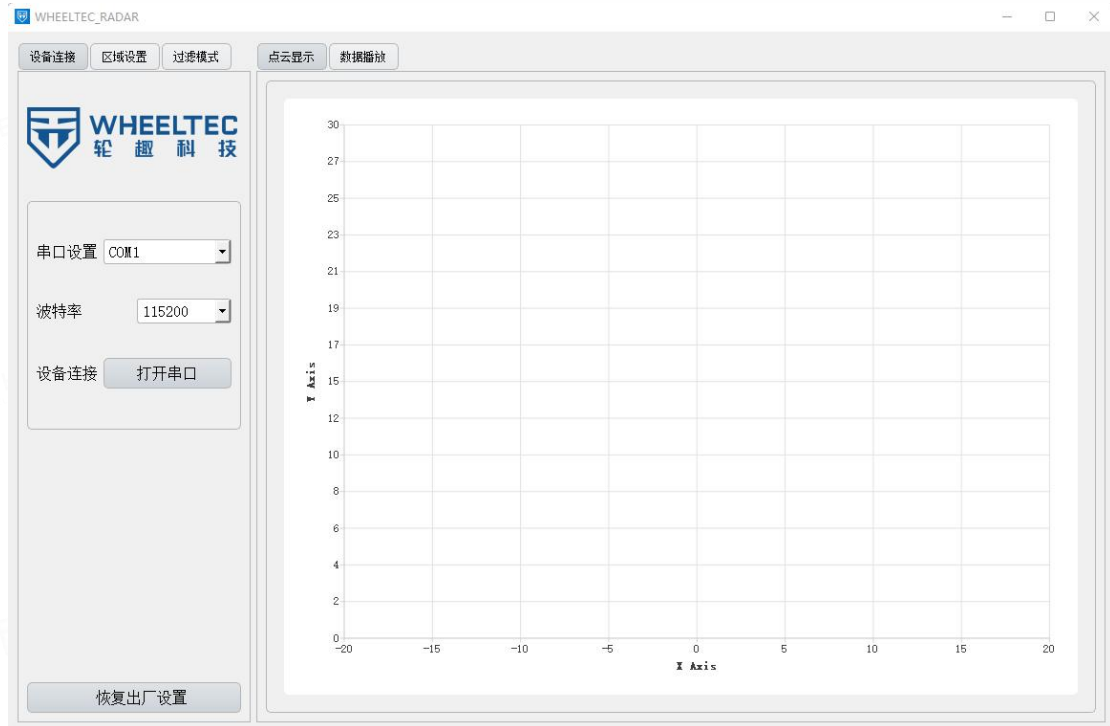
进入解压后的文件夹，找到 WHEELTEC\_RADAR.exe 双击运行。

MR20毫米波雷达客户资料V1.0\_20251001 > 3.Windows上位机 > WHEELTEC\_RADAR

| 名称                    | 修改日期            | 类型     | 大小        |
|-----------------------|-----------------|--------|-----------|
| iconengines           | 2025/9/16 19:52 | 文件夹    |           |
| imageformats          | 2025/9/16 19:52 | 文件夹    |           |
| platforms             | 2025/9/16 19:52 | 文件夹    |           |
| translations          | 2025/9/16 19:52 | 文件夹    |           |
| D3Dcompiler_47.dll    | 2014/3/11 18:54 | 应用程序扩展 | 3,386 KB  |
| libEGL.dll            | 2019/12/4 4:49  | 应用程序扩展 | 28 KB     |
| libgcc_s_dw2-1.dll    | 2015/12/29 6:25 | 应用程序扩展 | 118 KB    |
| libGLSV2.dll          | 2019/12/4 4:49  | 应用程序扩展 | 2,748 KB  |
| libstdc++-6.dll       | 2015/12/29 6:25 | 应用程序扩展 | 1,505 KB  |
| libwinpthread-1.dll   | 2015/12/29 6:25 | 应用程序扩展 | 78 KB     |
| opengl32sw.dll        | 2016/6/14 21:08 | 应用程序扩展 | 15,621 KB |
| Qt5Charts.dll         | 2019/12/4 6:06  | 应用程序扩展 | 1,379 KB  |
| Qt5Core.dll           | 2025/9/16 19:49 | 应用程序扩展 | 6,004 KB  |
| Qt5Gui.dll            | 2019/12/4 4:49  | 应用程序扩展 | 6,086 KB  |
| Qt5SerialPort.dll     | 2019/12/4 4:57  | 应用程序扩展 | 82 KB     |
| Qt5Svg.dll            | 2019/12/4 5:00  | 应用程序扩展 | 358 KB    |
| Qt5Widgets.dll        | 2019/12/4 4:49  | 应用程序扩展 | 6,219 KB  |
| <b>WHEELTEC_RADAR</b> | 2025/9/16 17:18 | 应用程序   | 133 KB    |

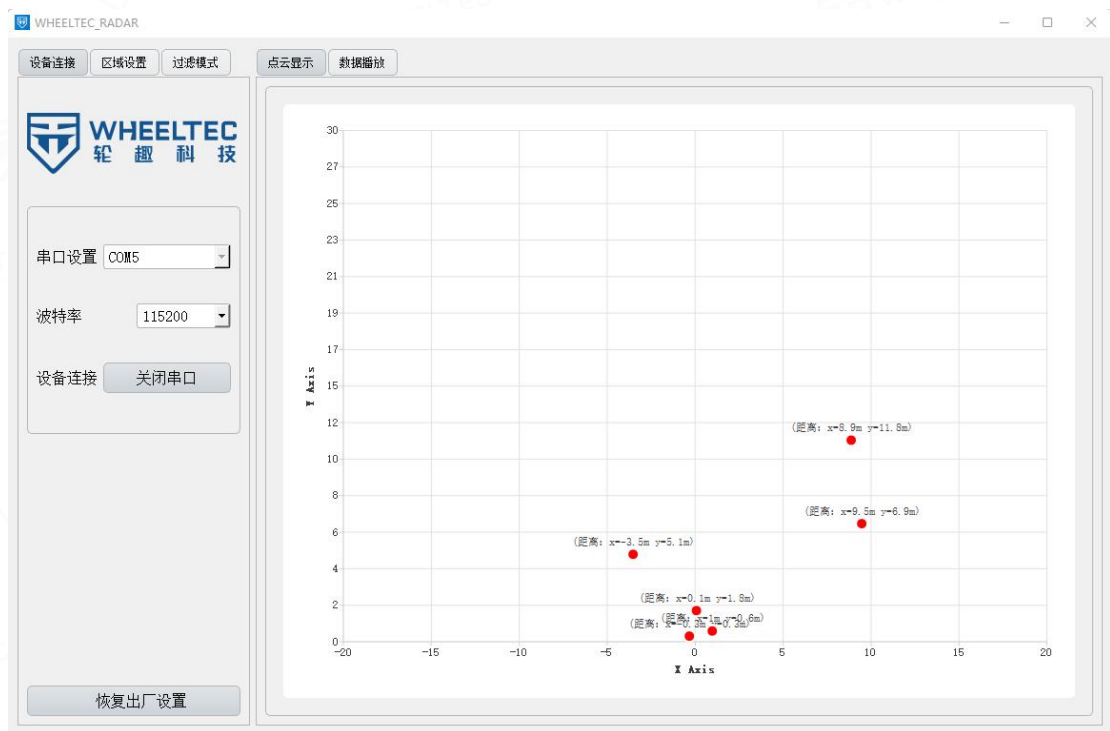
图 1-2-4 运行上位机软件

打开上位机软件后，选择毫米波雷达对应的串口号后，点击打开串口便能连上雷达。



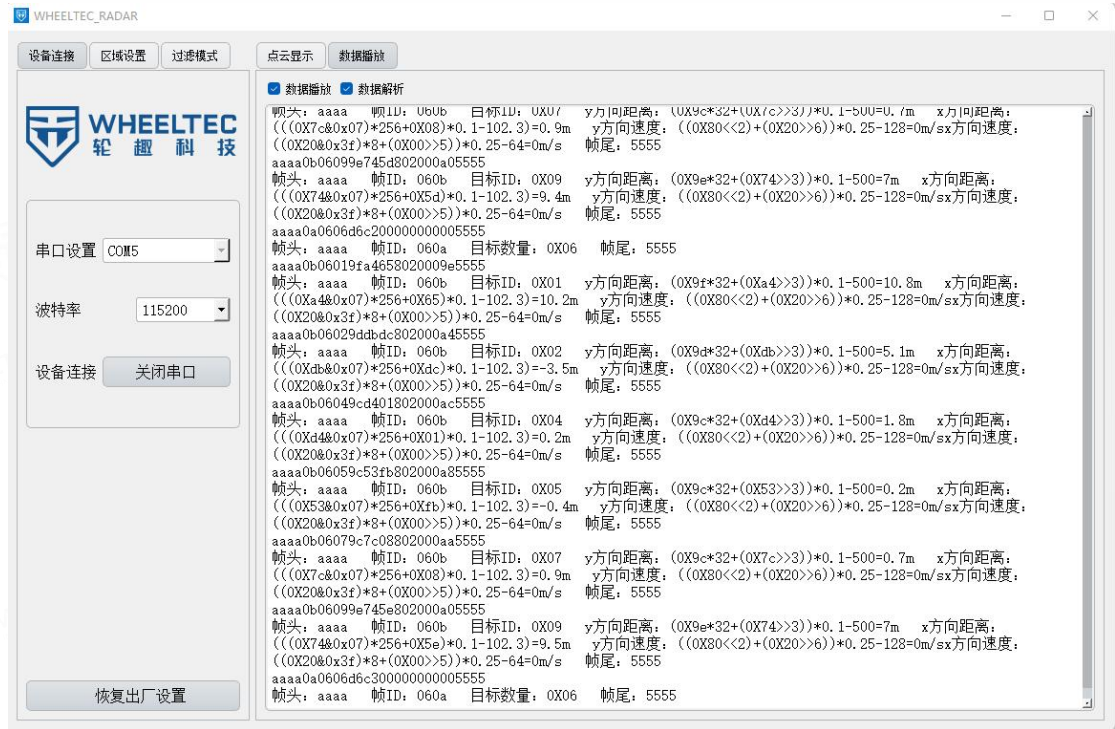
### ③ 上位机软件功能

#### 1) 毫米波雷达点云显示



#### 2) 毫米波雷达数据播放及解析

通过勾选数据播放或者数据解析开启该功能

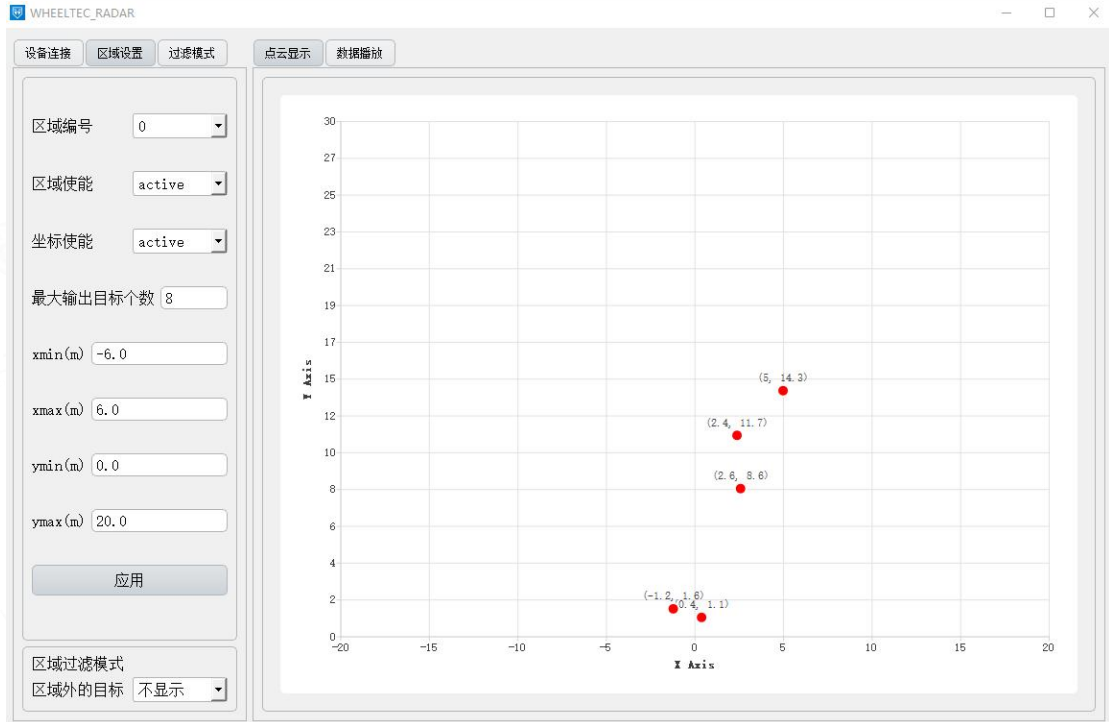


### 3) 区域设置

目前仅允许设置 3 个区域，设置矩形框内的允许输出目标个数为 N，如果 N 大于矩形框内的实际检测的目标个数，则输出实际检测的目标，如果 N 小于实际检测的目标个数，则输出 N 个距离最近的目标（按径向距离排序，由近及远），目前仅支持最大输出目标个数为 16 个。

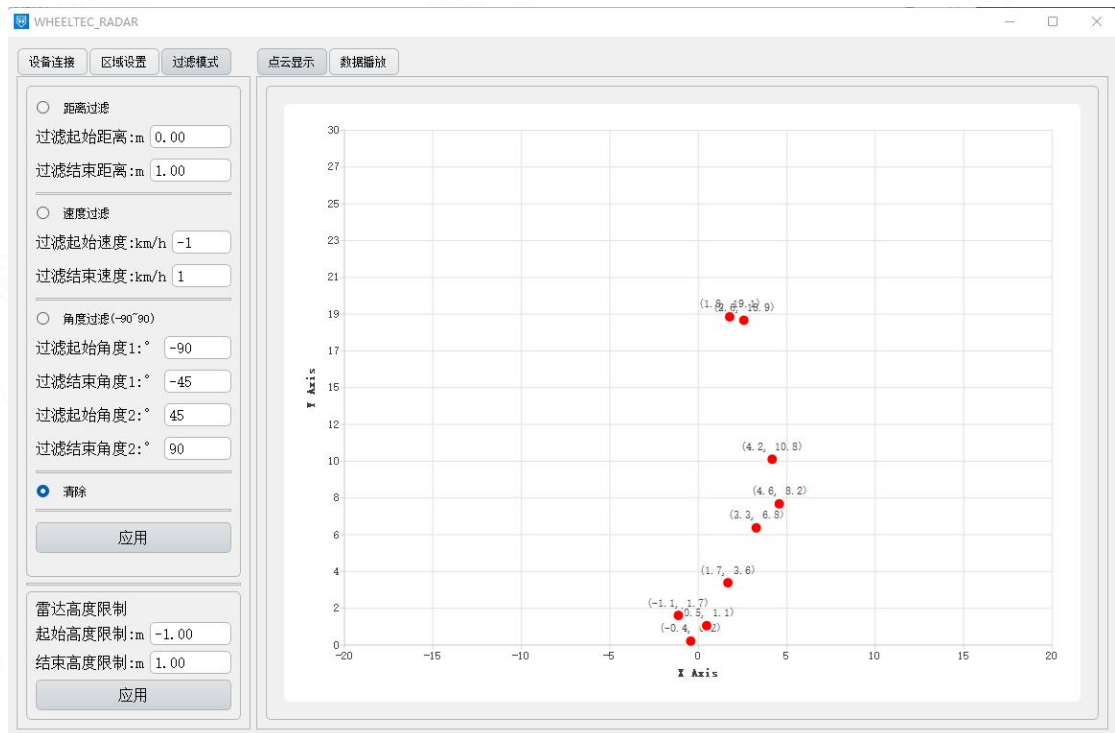
当区域使能及坐标使能同时为 active 时区域设置才会生效，否则不生效。

区域过滤功能需要配合区域设置功能，选择是否过滤掉区域设置之外的目标。



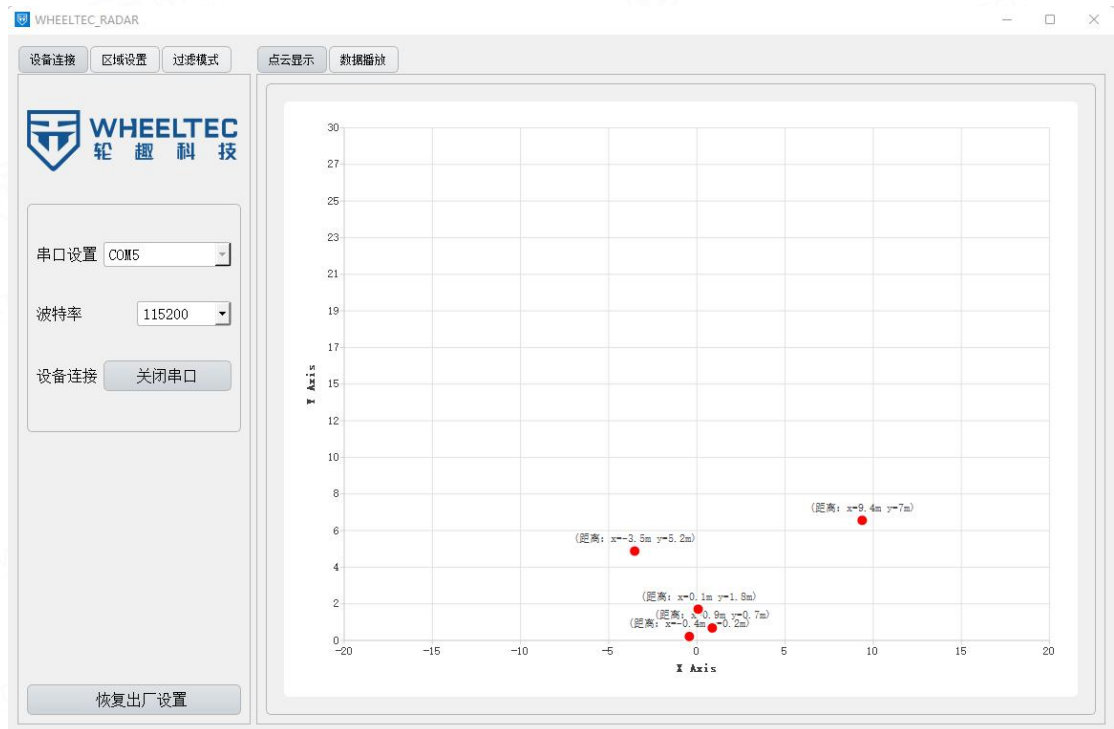
#### 4) 过滤模式

过滤模式有四种模式，不过滤、距离过滤、速度过滤、角度过滤。选择合适的过滤模式，就能过滤掉两个参数范围内的目标（其中角度过滤以雷达前方为0度角，角度顺时针方向增加）。



雷达高度限制：限制高度范围，超出此范围的目标不显示

## 5) 恢复出厂设置



按下恢复出厂设置按钮即可取消所有配置，恢复出厂状态

## 1.3 CH9102 信息修改

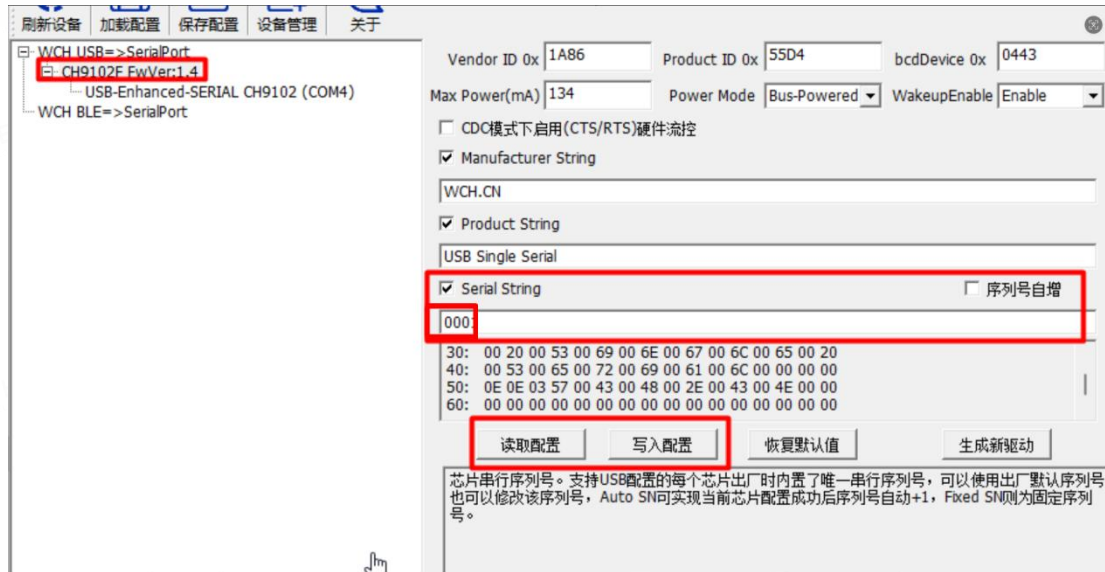
点进驱动软件及工具目录下的 CH9102 修改串口号工具目录，得到以下文件，双击运行 CH34xSerCfg.exe。

× MR20毫米波雷达客户资料V1.0\_20251001 > 6.驱动软件及工具 > CH9102修改串口号工具

| 名称                                                                                              | 修改日期           | 类型     | 大 |
|-------------------------------------------------------------------------------------------------|----------------|--------|---|
|  CH34xSerCfg | 2024/8/21 9:17 | 应用程序   |   |
|  CH343PT.dll | 2024/8/21 9:17 | 应用程序扩展 |   |

之后若我们的电脑有连接 CH9102 芯片的设备则会显示相关参数，我们可以通过修改对应参数栏后面的值并点击“写入配置”进行应用修改。我们毫米波雷达默认的 serial string 串口号为 0006。

若没有显示相关参数，serial string 为空，则点击恢复默认值在点击写入配置，上电重新连接即可。



## 2. Ubuntu

### 2.1 Ubuntu CH9102 驱动安装

可以通过在终端执行 `lsmod|grep ch343` 命令确认系统是否已经有该驱动模块。

#### ① 解压驱动压缩包

首先解压“ch343ser\_linux（CH9102 的 linux 驱动）.zip”到 Ubuntu 系统中。



图 2-1-1 解压驱动压缩包

#### ② 移除 cdc\_acm 模块

首先运行 `lsmod | grep cdc_acm` 检查是否有加载 `cdc_acm` 模块，此模块一般为 Linux 自带模块，可用于注册相应 USB 设备驱动等。如果显示已加载到内核的模块有 `cdc_acm`，则运行 `sudo rmmod cdc_acm` 先将其 `rmmod`，如果未显示相应模块则无需 `rmmod`。

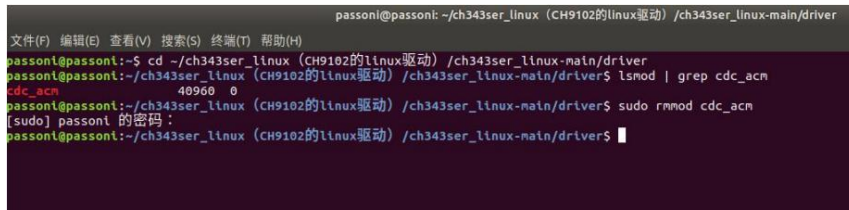


图 2-1-2 移除 cdc\_acm 说明

### ③ 安装驱动

在 driver 路径下终端输入 make 根据文件夹内的编译规则对.c 文件进行编译，会相应生成.ko 等文件如图 1-2-5:

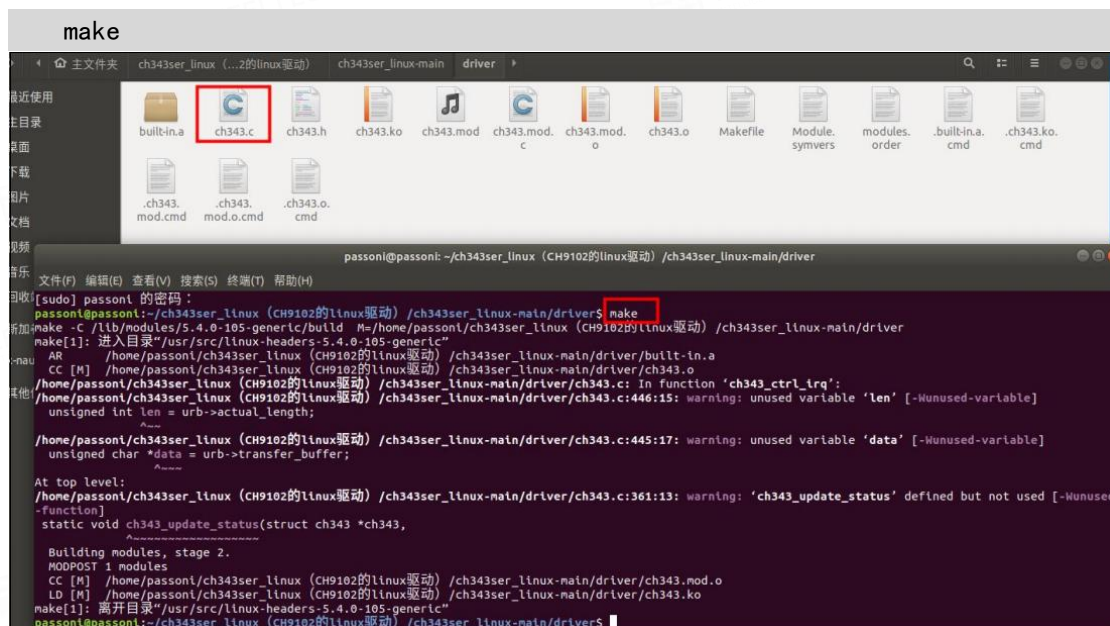


图 2-1-3 编译 ch343.c

完成编译后需要将模块载入，具体细节可参照 Makefile，这里不再展开

```
sudo make load
```

模块载入后再将文件安装到指定位置:

```
sudo make install
```

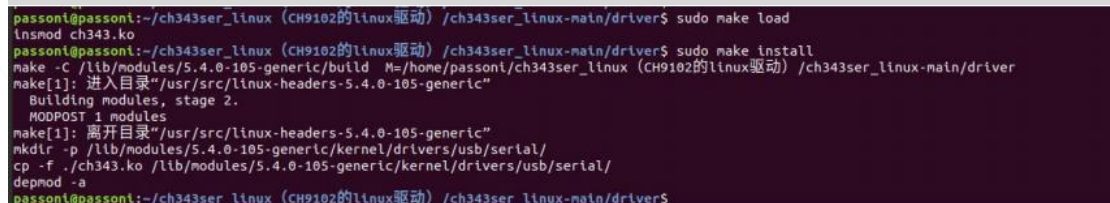


图 2-1-4 载入模块并安装

为了方便后续系统启动会自动加载 ch343.ko，需要在/etc/modules 文件里加入 ch343 在终端输入:

```
sudo gedit /etc/modules
```

新增 ch343 在最后一行后保存并退出重启系统，则系统会默认自动加载

ch343 驱动，重启后在终端输入命令查看是否自动加载：

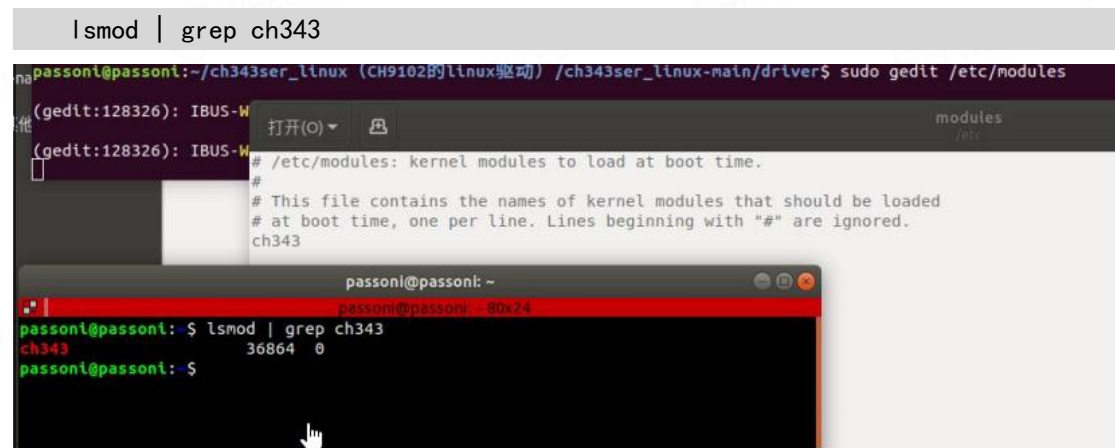
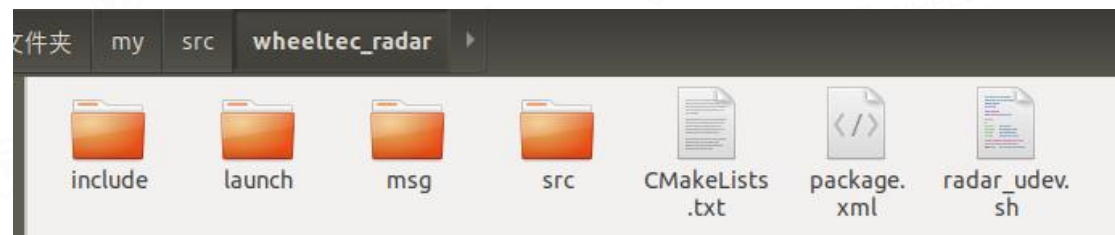


图 2-1-5 编辑/etc/modules 文件

## 2. 2ROS1\_SDK 包

### ① 解压 ROS1\_SDK 压缩包

进入 ROS1\_SDK 目录中，解压其中的压缩包，并将我们刚刚解压得到的 wheeltec\_radar 包复制到 ros 工作空间的 src 目录中。



### ② 运行终端并编译

配置毫米波雷达串口规则

```
sudo sh ~/my/src/wheeltec_radar/radar_udev.sh
```

运行以下命令编译功能包。

```
catkin_make -DCATKIN_WHITELIST_PACKAGES=wheeltec_radar
```

```
-- Found gtest sources under '/usr/src/gtest': gtests will be built
-- Found gmock sources under '/usr/src/gtest': gmock will be built
-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
-- Looking for pthread.h
-- Looking for pthread.h - found
-- Looking for pthread_create
-- Looking for pthread_create - not found
-- Looking for pthread_create in pthreads
-- Looking for pthread_create in pthreads - not found
-- Looking for pthread_create in pthread
-- Looking for pthread_create in pthread - found
-- Found Threads: TRUE
-- Using Python nosetests: /usr/bin/nosetests-2.7
-- catkin 0.7.29
-- BUILD_SHARED_LIBS is on
-- BUILD_SHARED_LIBS is on
-- Using CATKIN_WHITELIST_PACKAGES: wheeltec_radar
--
-- traversing 1 packages in topological order:
--   - wheeltec_radar
--
-- +++ processing catkin package: 'wheeltec_radar'
==> add_subdirectory(wheeltec_radar)
-- Using these message generators: gencpp;geneus;genlisp;gennodejs;genpy
-- Configuring done
-- Generating done
-- Build files have been written to: /home/wheeltec-client/my/build
####
#### Running command: "make -j2 -l2" in "/home/wheeltec-client/my/build"
####
Scanning dependencies of target wheeltec_radar_node
[ 50%] Building CXX object wheeltec_radar/CMakeFiles/wheeltec_radar_node.dir/src/wheeltec_radar.cpp.o
[100%] Linking CXX executable /home/wheeltec-client/my/devel/lib/wheeltec_radar/wheeltec_radar_node
[100%] Built target wheeltec_radar_node
wheeltec-client@ubuntu:~/my$ catkin_make -DCATKIN_WHITELIST_PACKAGES=wheeltec_radar
```

图 2-2-2 编译成功

### ③ 将毫米波雷达连接至 Ubuntu 系统

至此已经安装完 CH9102 的 Linux 驱动，安装完成后记得重启系统检查设备能否正常加载，在终端输入命令可查看设备已识别成 wheeltec\_mmwave\_radar。

```
ll /dev
```

|            |        |      |     |     |              |                                  |
|------------|--------|------|-----|-----|--------------|----------------------------------|
| crw-rw-rw- | 1 root | root | 10, | 59  | 10月 31 02:21 | vmctl                            |
| crw-rw-rw- | 1 root | root | 10, | 58  | 10月 31 02:21 | vsocok                           |
| lrwxrwxrwx | 1 root | root |     | 7   | 10月 31 02:24 | wheeltec_mmwave_radar -> ttyACM0 |
| crw-rw-rw- | 1 root | root | 1,  | 5   | 10月 31 02:21 | zero                             |
| crw-rw-rw- | 1 root | root | 10, | 249 | 10月 31 02:21 | zfs                              |

```
wheeltec-client@ubuntu:~$ ll /dev
```

图 2-2-3 查看已连接设备

注：如果需要创建串口别名的，可以参考功能包中的“radar\_udev.sh”脚本。本质是通过设备属性来定位设备并为其创建命名规则。

### ④ 配置驱动包

打开功能包下的 wheeltec\_radar.launch，将 usart\_port\_name 参数修改为毫米波雷达设备号（ll /dev 查看）

```

wheeltec_radar.launch
~/my/src/wheeltec_radar/launch

<launch>
  <arg name="usart_port_name" default="/dev/wheeltec_mmwave_radar" />
  <arg name="serial_baud_rate" default="115200" />

  <node name="wheeltec_radar_node" pkg="wheeltec_radar" type="wheeltec_radar_node" output="screen">
    <param name="usart_port_name" value="$(arg usart_port_name)" />
    <param name="serial_baud_rate" value="$(arg serial_baud_rate)" />
  </node>

  <!-- 静态 TF 变换: base_footprint -> radar -->
  <!-- <node pkg="tf2_ros" type="static_transform_publisher" name="base_to_radar" args="0 0 0 0 0 0
base_footprint radar" /> -->
</launch>

```

图 2-2-4 查看 wheeltec\_radar.launch

## ⑤ 启动毫米波雷达

在配置完 launch 文件后，我们打开一个终端，执行以下命令：

```
roslaunch wheeltec_radar wheeltec_radar.launch
```

成功启动后终端会出现以下字样。

```

wheeltec-client@ubuntu:~/my$ roslaunch wheeltec_radar wheeltec_radar.launch
... logging to /home/wheeltec-client/.ros/log/a3993670-9468-11f0-8ec2-000c298f917a/roslaunch-ubuntu-3412.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.0.136:38263/

SUMMARY
=====

PARAMETERS
* /roscdistro: melodic
* /rosversion: 1.14.13
* /wheeltec_radar_node/serial_baud_rate: 115200
* /wheeltec_radar_node/usart_port_name: /dev/wheeltec_lidar

NODES
/
  base_to_radar (tf2_ros/static_transform_publisher)
  wheeltec_radar_node (wheeltec_radar/wheeltec_radar_node)

auto-starting new master
process[master]: started with pid [3423]
ROS_MASTER_URI=http://192.168.0.136:11311

setting /run_id to a3993670-9468-11f0-8ec2-000c298f917a
process[rosout-1]: started with pid [3434]
started core service [/rosout]
process[wheeltec_radar_node-2]: started with pid [3437]
process[base_to_radar-3]: started with pid [3438]
[ INFO] [1758183763.954122714]: wheeltec_radar serial port opened

```

图 2-2-5 成功启动

## ⑥ 雷达数据查看

当启动雷达后，/wheeltec\_radar\_node 节点便会在 ros 中发布一个/radarscan 话题和/radarpointcloud 话题以及/radartrack 话题。

```
wheeltec-client@ubuntu:~/my$ rostopic list
/radarpointcloud
/radarscan
/radartrack
/rosout
/rosout_agg
wheeltec-client@ubuntu:~/my$ rostopic info /radarscan
Type: wheeltec_radar/RadarDetectionArray

Publishers:
* /wheeltec_radar_node (http://192.168.0.136:36773/)

Subscribers: None

wheeltec-client@ubuntu:~/my$ rostopic info /radarpointcloud
Type: sensor_msgs/PointCloud2

Publishers:
* /wheeltec_radar_node (http://192.168.0.136:36773/)

Subscribers: None
```

图 2-2-6 查看相关话题发布者

我们可以通过执行 `rostopic echo /radarscan` 命令直接进行毫米波识别目标的查看。`/radarscan` 话题中的消息类型是 `wheeltec_radar/RadarDetectionArray`，该消息类型的具体描述如下图

```
wheeltec-client@ubuntu:~/my$ rosmmsg show wheeltec_radar/RadarDetectionArray
std_msgs/Header header
  uint32 seq
  time stamp
  string frame_id
wheeltec_radar/RadarDetection[] detections
  uint16 detection_id
  geometry_msgs/Point position
    float64 x
    float64 y
    float64 z
  geometry_msgs/Vector3 velocity
    float64 x
    float64 y
    float64 z
  float64 amplitude
```

图 2-2-7 话题数据格式

除了直接通过话题查看文本形式的雷达数据，我们也可以通过 `rviz` 来查看毫米波识别目标的图像。直接在终端输入 `rviz` 并执行。打开 `rviz` 后，先将 Fixed Frame 后面对应的值修改为毫米波雷达 `frame_id`: `radar`。

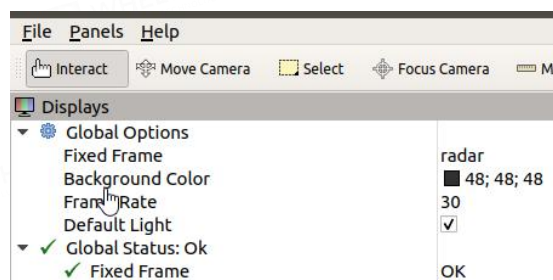


图 2-2-8 修改 Fixed Frame

之后点击 Rviz 左下角的 Add 按钮，在弹出的窗口中点击 By topic 选中 /radarpointcloud 话题并点击 OK。

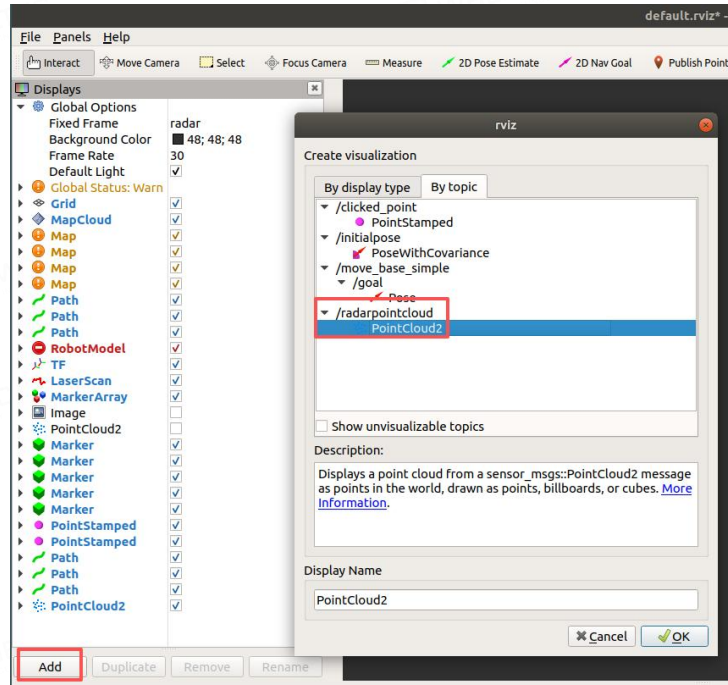


图 2-2-9 添加 radarpointcloud

成功添加后，修改/radarpointcloud 话题的目标点云大小 size 为 0.5，我们便可以在 Rviz 中看到这样的毫米波目标点云图像。

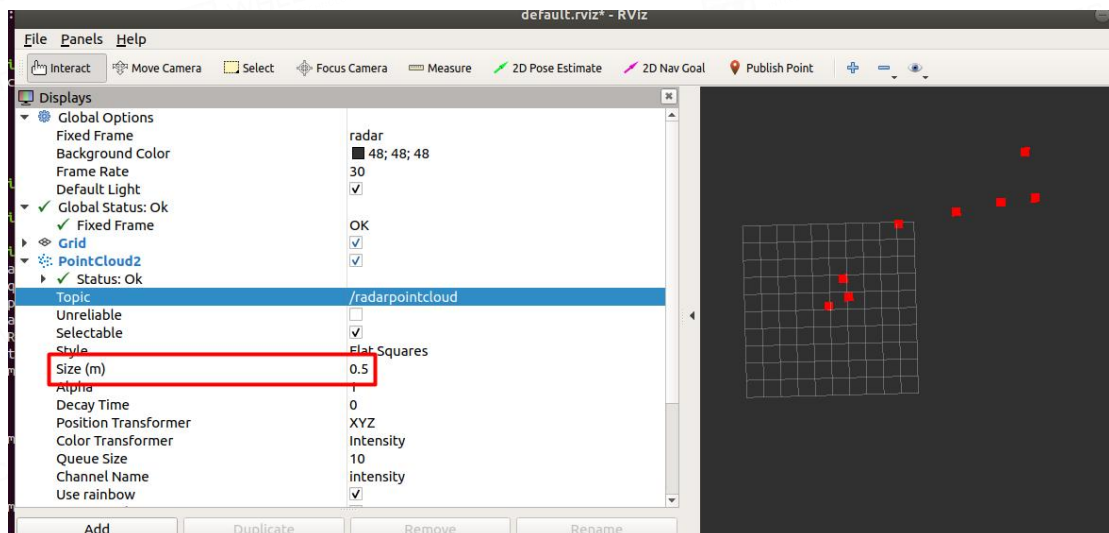


图 2-2-10 Rviz 显示目标点云

## 2.3 ROS2\_SDK 包

### ① 解压 ROS2\_SDK 压缩包

进入 ROS2\_SDK 目录中，解压其中的压缩包，并将我们刚刚解压得到的 wheeltec\_radar 包复制到 ros2 工作空间的 src 目录中。

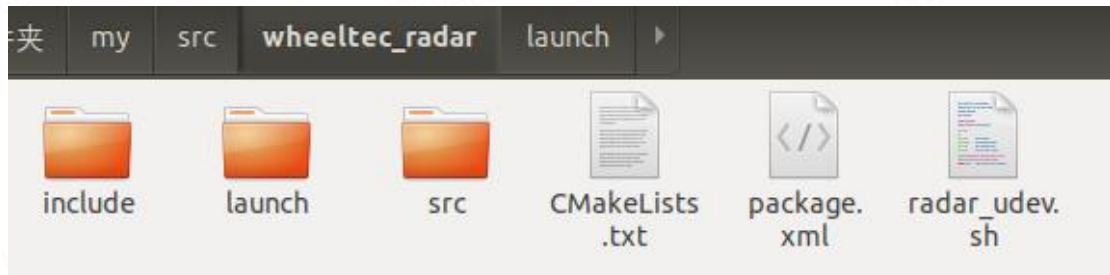


图 2-3-1 wheeltec\_radar 放置到 ros 工作空间

## ② 运行终端并编译

安装依赖

```
sudo apt-get install ros-humble-radar-msgs
```

配置毫米波雷达串口规则

```
sudo sh ~/my/src/wheeltec_radar/radar_udev.sh
```

运行以下命令编译功能包(如果报错缺少 serial 依赖，请将 ROS2\_SDK 目录中 serial\_ros2 功能包放在 src 目录中一起编译)。

```
colcon build

wheeltec@humble:~/ws$ colcon build
Starting >>> serial
--- stderr: serial
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc: In function 'std::vector<std::__cxx11::basic_string<char> > glob(const std::vector<std::__cxx11::basic_string<char> >&)':
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc:66:40: warning: comparison of integer expressions of different signedness: 'int' and 'size_t' {aka 'long unsigned int'} [-Wsign-compare]
   66 |     for(int path_index = 0; path_index < glob_results.gl_pathc; path_index++)
      |                               ~~~~~^~~~~
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc:57:9: warning: variable 'glob_retval' set but not used [-Wunused-but-set-variable]
   57 |     int glob_retval = glob(patterns[0].c_str(), 0, NULL, &glob_results);
      |     ~~~~~^~~~~
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc: In function 'std::string format(const char*, ...)':
/home/wheeltec/ws/src/serial_ros2/src/impl/list_ports/list_ports_linux.cc:246:31: warning: comparison of integer expressions of different signedness: 'int' and 'size_t' {aka 'long unsigned int'} [-Wsign-compare]
   246 |         else if( return_value >= buffer_size_bytes )
      |                      ~~~~~^~~~~
---
Finished <<< serial [5.20s]
Starting >>> wheeltec_radar
--- stderr: wheeltec_radar
/home/wheeltec/ws/src/wheeltec_radar/src/wheeltec_radar.cpp: In member function 'void turn_on_radar::Get_Data()':
/home/wheeltec/ws/src/wheeltec_radar/src/wheeltec_radar.cpp:81:57: warning: suggest parentheses around '+' inside '<<' [-Wparentheses]
   81 |         radarreturn.doppler_velocity = (Receive_Data[8]<<2+(Receive_Data[9]>>6))*0.25-128;
      |                                     ~~~~~^~~~~
/home/wheeltec/ws/src/wheeltec_radar/src/wheeltec_radar.cpp:90:50: warning: suggest parentheses around '+' inside '<<' [-Wparentheses]
   90 |         out_object.velocity.x = (Receive_Data[8]<<2+(Receive_Data[9]>>6))*0.25-128;
      |                                     ~~~~~^~~~~
---
Finished <<< wheeltec_radar [28.9s]

Summary: 2 packages finished [34.4s]
2 packages had stderr output: serial wheeltec_radar
```

图 2-3-2 编译成功

### ③ 将毫米波雷达连接至 Ubuntu 系统

至此已经安装完 CH9102 的 Linux 驱动，安装完成后记得重启系统检查设备能否正常加载，在终端输入命令可查看设备已识别成 wheeltec\_mmwave\_radar。

```
ll /dev
```

```
crw-rw-rw- 1 root root 10, 59 10月 31 02:21 vmecl
crw-rw-rw- 1 root root 10, 58 10月 31 02:21 vsock
lrwxrwxrwx 1 root root 7 10月 31 02:24 wheeltec_mmwave_radar -> ttyACM0
crw-rw-rw- 1 root root 1, 5 10月 31 02:21 zero
crw-rw-rw- 1 root root 10, 249 10月 31 02:21 zfs
```

```
wheeltec-client@ubuntu:~$ ll /dev
```

图 2-3-3 查看已连接设备

注：如果需要创建串口别名的，可以参考功能包中的“radar\_udev.sh”脚本。本质是通过设备属性来定位设备并为其创建命名规则。

### ④ 配置驱动包

打开功能包下的 wheeltec\_radar.launch.py，将 usart\_port\_name 参数修改为毫米波雷达设备号（ll /dev 查看），修改完需要重新编译一下。

```
打开(O) wheeltec_radar.launch.py
~/my/src/wheeltec_radar
```

```
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, GroupAction, IncludeLaunchDescription
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
from launch_ros.actions import Node
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

#def launch(launch_descriptor, argv):
def generate_launch_description():

    # Extract common parameters
    common_params = {
        'usart_port_name': '/dev/wheeltec_mmwave_radar',
        'serial_baud_rate': 115200,
    }

    turn_on_radar = Node(
        package='wheeltec_radar',
        executable='wheeltec_radar_node',
        name='wheeltec_radar_node',
        output='screen',
        parameters=[common_params],
    )

    base_to_radar = Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        name='base_to_radar',
        arguments=['0', '0', '0', '0', '0', '0', 'base_footprint', 'radar'],
    )

    ld = LaunchDescription()
    ld.add_action(turn_on_radar)
    #ld.add_action(base_to_radar)

    return ld
```

图 2-3-4 查看 wheeltec\_radar.launch.py

### ⑤ 启动毫米波雷达

在配置完 launch 文件后，我们打开一个终端，执行以下命令：

```
ros2 launch wheeltec_radar wheeltec_radar.launch.py
```

成功启动后终端会出现以下字样。

```
wheeltec@humble:~/ws$ ros2 launch wheeltec_radar wheeltec_radar.launch.py
[INFO] [launch]: All log files can be found below /home/wheeltec/.ros/log/2025-09-18-17-50-30-527259-humble-4344
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [wheeltec_radar_node-1]: process started with pid [4345]
[INFO] [static_transform_publisher-2]: process started with pid [4347]
[static_transform_publisher-2] [WARN] [1758189031.283456501] []: Old-style arguments are deprecated; see --help for new-style arguments
[static_transform_publisher-2] [INFO] [1758189031.591180815] [base_to_radar]: Spinning until stopped - publishing transform
[static_transform_publisher-2] translation: ('0.000000', '0.000000', '0.000000')
[static_transform_publisher-2] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-2] from 'base_footprint' to 'radar'
[wheeltec_radar_node-1] [INFO] [1758189031.831777797] [wheeltec_radar_node]: wheeltec_radar serial port opened
```

图 2-3-5 成功启动

### ⑥ 雷达数据查看

当启动雷达后，/wheeltec\_radar\_node 节点便会在 ros 中发布一个/radarscan 话题和/radarpointcloud 话题以及/radartrack 话题。

```
wheeltec@humble:~/ws$ ros2 topic list
/parameter_events
/radarpointcloud
/radarscan
/radartrack
/rosout
/tf_static
wheeltec@humble:~/ws$ ros2 topic info /radarscan
Type: radar_msgs/msg/RadarScan
Publisher count: 1
Subscription count: 0
wheeltec@humble:~/ws$ ros2 topic info /radarpointcloud
Type: sensor_msgs/msg/PointCloud2
Publisher count: 1
Subscription count: 0
```

图 2-3-6 查看相关话题发布者

我们可以通过执行 `ros2 topic echo /radarscan` 命令直接进行毫米波识别目标的查看。/radarscan 话题中的消息类型是 `radar_msgs/msg/RadarScan`，该消息类型的具体描述如下图

```
wheeltec@humble:~/ws$ ros2 interface show radar_msgs/msg/RadarScan
std_msgs/Header header
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id

radar_msgs/RadarReturn[] returns
  float32 range #
  float32 azimuth #
  # Positive angles are anticlockwise from the sensor and negative angles clockwise from the sensor as per REP-0103.
  float32 elevation #
  # Negative angles are below the sensor. For 2D radar, this will be 0.
  float32 doppler_velocity #
  float32 amplitude #
```

图 2-3-7 话题数据格式

除了直接通过话题查看文本形式的雷达数据，我们也可以通过 rviz2 来查看毫米波识别目标的图像。直接在终端输入 `rviz2` 并执行。打开 `rviz2` 后，先将 Fixed Frame 后面对应的值修改为毫米波雷达 `frame_id: radar`。

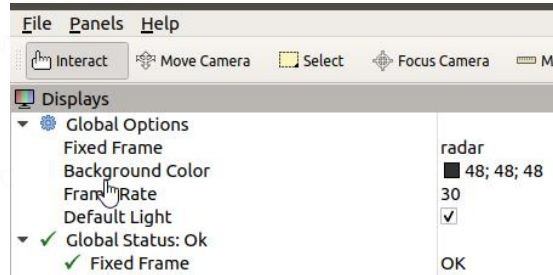


图 2-3-8 修改 Fixed Frame

之后点击 Rviz2 左下角的 Add 按键，在弹出的窗口中点击 By topic 选中 `/radarpointcloud` 话题并点击 OK。

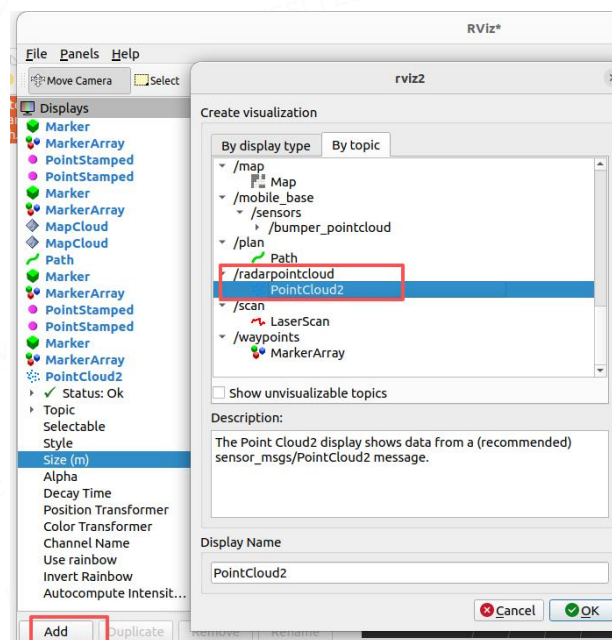


图 2-3-9 添加 radarpointcloud

成功添加后，修改 `/radarpointcloud` 话题的目标点云大小 `size` 为 0.5，我们便可以在 Rviz2 中看到这样的毫米波目标点云图像。

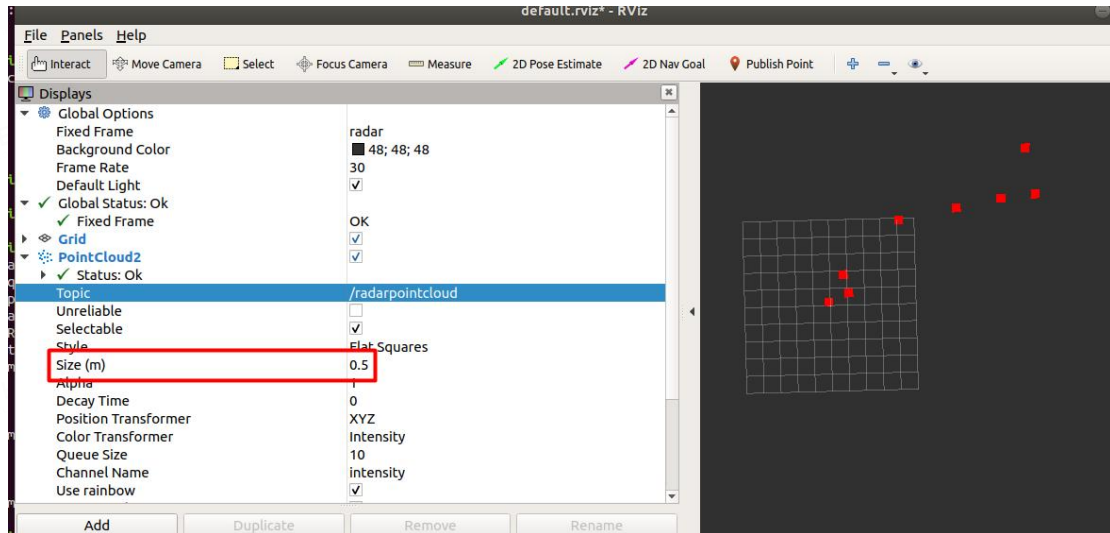


图 2-3-10 Rviz2 显示目标点云